

Отчёт по лабораторной работе №3

Компьютерный практикум по статистическому анализу

Арбатова Варвара Петровна

В данном отчёте представлены результаты выполнения лабораторной работы по теме “Управляющие структуры”

Описаны использованные методы, полученные экспериментальные данные и их анализ.

Сделаны выводы о достижении поставленной цели.

Содержание

1	Введение	7
1.1	Актуальность	7
1.2	Цель работы	7
1.3	Задачи	7
1.4	Объект и предмет исследования	8
1.5	Методы исследования	8
1.6	Схема выполнения работы	8
2	Теоретическая часть	10
2.1	Циклы while и for	10
2.2	Условные операторы	11
2.3	Функции	12
2.4	Функции высшего порядка	13
2.5	Работа с матрицами	13
3	Материалы и методы	15
4	Выполнение лабораторной работы	16
4.1	Задание 1. Числа от 1 до 100 и их квадраты	16
4.2	Задание 2. Условный и тернарный операторы	17
4.3	Задание 3. Функция add_one	18
4.4	Задание 4. map / broadcast для матрицы A	19
4.5	Задание 5. Матрица A, A ³ и замена столбца	19
4.6	Задание 6. Матрица B и C = B⊗B	20
4.7	Задание 7. Матрицы Z ₁ –Z ₄ размера 6×6	21
4.8	Задание 8. Функция outer(x, y, op)	24
4.9	Задание 9. Решение системы линейных уравнений	27
4.10	Задание 10. Анализ матрицы M 6×10	29
4.11	Задание 11. Вычисление двойных сумм	31
5	Результаты	33
6	Обсуждение результатов	35
7	Заключение	36
	Список литературы	37

A	Приложение А. Полный листинг программы	38
B	Приложение Б. Использованные функции и конструкции Julia	46

Список иллюстраций

1.1	Блок-схема выполнения лабораторной работы	9
-----	---	---

Список таблиц

1 Введение

1.1 Актуальность

Язык Julia сочетает простоту синтаксиса, характерную для Python, с производительностью компилируемых языков. Он широко применяется в научных вычислениях, анализе данных и машинном обучении. Освоение управляющих конструкций и работы с матрицами — базовая навыковая часть подготовки специалиста по прикладной математике и информатике.

1.2 Цель работы

Основная цель работы — освоить применение циклов, функций и сторонних пакетов Julia для решения задач линейной алгебры и работы с матрицами.

1.3 Задачи

1. Освоить циклы `while` и `for`.
2. Освоить условные операторы (`if-elseif-else`, тернарный оператор).
3. Освоить способы объявления функций (полный, однострочный, анонимный).
4. Изучить функции высшего порядка `map` и `broadcast`.
5. Научиться работать с матрицами: создание, индексация, срезы, умножение, решение линейных систем.
6. Выполнить 11 заданий, приведённых в методических указаниях.

1.4 Объект и предмет исследования

Объект: язык программирования Julia и его управляющие конструкции.

Предмет: применение циклов, функций и операций линейной алгебры для решения прикладных задач.

1.5 Методы исследования

Экспериментально-расчётные методы: написание и исполнение программ на языке Julia, анализ результатов.

1.6 Схема выполнения работы

На рис. 1.1 представлена общая последовательность действий.

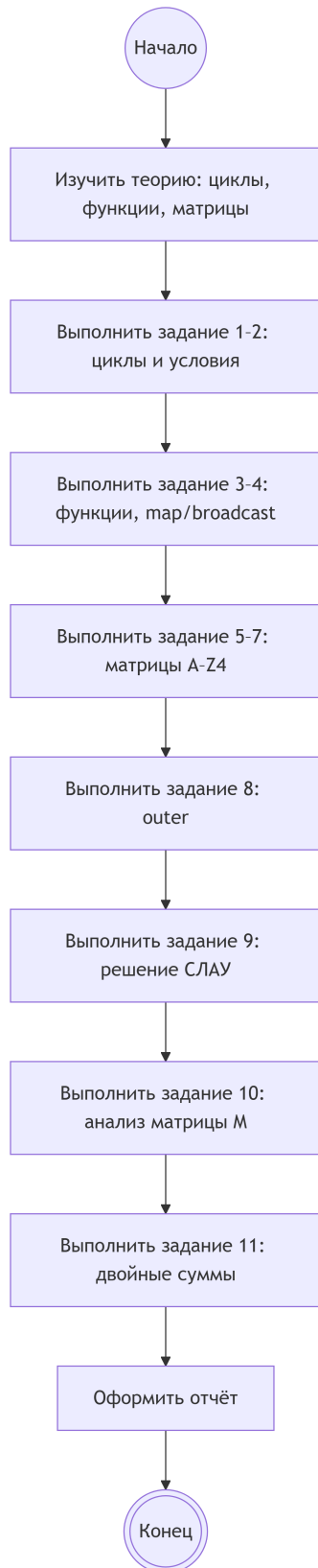


Рисунок 1.1: Блок-схема выполнения лабораторной работы

2 Теоретическая часть

2.1 Циклы **while** и **for**

Julia предоставляет два основных вида циклов.

Цикл **while**:

```
while <[выражение]>  
    <[блок кода]>  
end
```

Пример: вывод чисел от 1 до 10.

```
n = 0  
while n < 10  
    n += 1  
    println(n)  
end
```

Цикл **for**:

```
for <[диапазон]> in <[коллекция]>  
    <[блок кода]>  
end
```

Пример: вывод нечётных чисел от 1 до 10.

```

for n in 1:2:10
    println(n)
end

```

Цикл `for` также позволяет перебирать элементы массива напрямую, без индексов:

```

for friend in ["Ted", "Robyn", "Barney"]
    println("Hi $friend, it's great to see you!")
end

```

Двойной цикл можно записать компактно:

```

for i in 1:m, j in 1:n
    A[i, j] = i + j
end

```

2.2 Условные операторы

Полная форма:

```

if <XXXXXXXX 1>
    <XXXXXXXX 1>
elseif <XXXXXXXX 2>
    <XXXXXXXX 2>
else
    <XXXXXXXX 3>
end

```

Тернарный оператор:

```
a ? b : c
```

эквивалентен:

```
if a
    b
else
    c
end
```

Пример:

```
x = 5
y = 10
(x > y) ? x : y    # 10
```

Важно: пробелы вокруг ? и : обязательны, иначе Julia не распознает тернарный оператор.

2.3 Функции

В Julia есть три способа объявления функции.

1. Полный синтаксис:

```
function sayhi(name)
    println("Hi $name, it's great to see you!")
end
```

2. Однострочный:

```
sayhi2(name) = println("Hi $name, it's great to see you!")  
f2(x) = x^2
```

3. Анонимный:

```
sayhi3 = name -> println("Hi $name, it's great to see you!")  
f3 = x -> x^2
```

По соглашению функции с восклицательным знаком (`sort!`) изменяют свой аргумент, а без него (`sort`) — возвращают новое значение.

2.4 Функции высшего порядка

map(f, collection) применяет функцию f к каждому элементу коллекции и возвращает массив результатов:

```
f(x) = x^2
map(f, [1, 2, 3])    # [1, 4, 9]
map(x -> x^3, [1, 2, 3])    # [1, 8, 27]
```

broadcast(f, collection) — обобщение tap, приводящее аргументы к общему измерению. Эквивалентная точечная запись:

$$f.([1, 2, 3])$$

Для двух аргументов:

a . + b #

2.5 Работа с матрицами

- `zeros(Int, m, n)` — матрица из нулей;

- `ones(Int, m, n)` — матрица из единиц;
- `fill(v, m, n)` — матрица, заполненная значением v ;
- `A[i, j]` — доступ к элементу;
- `A[:, j]` — срез по столбцу;
- `A[i, :]` — срез по строке;
- `A'` — транспонирование (сопряжённое);
- `A * B` — матричное умножение;
- `A^3` — возведение в степень;
- `A \ b` — решение системы $Ax = b$.

3 Материалы и методы

- **Оборудование:** ПК.
- **Программное обеспечение:** Julia 1.12, Jupyter Notebook.
- **Пакеты:** Random (для воспроизводимости случайных чисел).
- **Метод:** написание скриптов, исполнение в Jupyter, анализ результатов.

4 Выполнение лабораторной работы

4.1 Задание 1. Числа от 1 до 100 и их квадраты

Условие: используя циклы `while` и `for`, вывести целые числа от 1 до 100 и их квадраты; создать словарь `squares` и массив `squares_arr` с квадратами чисел.

4.1.1 Вывод через `for`

```
for i in 1:100
    println("$i - ", i^2)
end
```

Функция `println` печатает значения через запятую; `$i` — интерполяция переменной `i` в строку; `i^2` — возведение в степень.

4.1.2 Словарь `squares`

```
squares = Dict{Int,Int}()
for i in 1:100
    squares[i] = i^2
end
println(length(squares))    # 100
println(squares[1])        # 1
```

```
println(squares[50])      # 2500
println(squares[100])     # 10000
```

`Dict{Int,Int}()` создаёт пустой словарь с целочисленными ключами и значениями. `squares[i] = i^2` добавляет пару ключ–значение. `length` возвращает количество пар.

4.1.3 Массив `squares_arr` через `while`

```
i = 1
squares_arr = Int[]
while i <= 100
    push!(squares_arr, i^2)
    i += 1
end
println(length(squares_arr))  # 100
println(squares_arr[1])      # 1
println(squares_arr[50])     # 2500
println(squares_arr[100])    # 10000
```

`Int[]` создаёт пустой типизированный вектор; `push!(v, x)` добавляет элемент в конец массива (восклицательный знак — по соглашению об изменении аргумента).

4.2 Задание 2. Условный и тернарный операторы

Условие: напечатать число, если оно чётное, и строку «нечётное», если нечётное. Переписать через тернарный оператор.

4.2.1 Через **if-else**

```
a = 4
if (a % 2 == 0)
    println(a)
else
    println("XXXXXX")
end
```

$a \% 2$ — остаток от деления на 2.

4.2.2 Через тернарный оператор

```
x = 5
println(x % 2 == 0 ? x : "XXXXXX")
```

4.3 Задание 3. Функция **add_one**

```
add_one(x) = x + 1

println(add_one(5))    # 6
println(add_one(0))    # 1
println(add_one(-1))   # 0
```

Однострочный синтаксис $f(x) = \text{XXXXXX}$ эквивалентен полной форме с `function ... end`.

4.4 Задание 4. `map` / `broadcast` для матрицы `A`

```
A = [1 1 3; 5 2 6; -2 -1 -3]

A_plus1_map = map(add_one, A)
A_plus1_bc  = add_one.(A)
A_plus1     = A .+ 1

println("map:      ", A_plus1_map)
println("broadcast: ", A_plus1_bc)
println("A .+ 1:    ", A_plus1)
```

Все три способа дают один результат:

3×3 Matrix{Int64}:

```
 2  2  4
 6  3  7
-1  0 -2
```

`map` применяет функцию поэлементно; `add_one.(A)` — точечный вызов (синтаксический сахар над `broadcast`); `A .+ 1` — поэлементное сложение с числом (broadcasting скаляра).

4.5 Задание 5. Матрица `A`, `A3` и замена столбца

```
A = [1 1 3;
      5 2 6;
      -2 -1 -3]
```

```

A3 = A^3
display(A3)

A_mod = copy(A)
A_mod[:, 3] = A[:, 2] .+ A[:, 3]
display(A_mod)

```

Результаты:

```

A^3 =
3×3 Matrix{Int64}:
-18  -6  -30
-27  -9  -45
 27   9   45

```

```

A 3×3 Matrix{Int64}:
 1  1  4
 5  2  8
-2 -1 -4

```

- A^3 — матричное возведение в степень (не поэлементное!).
- `copy(A)` создаёт независимую копию, чтобы не портить исходную A .
- $A[:, 2]$ — второй столбец как вектор.
- `.+` — поэлементная сумма двух векторов.
- $A_mod[:, 3] = \dots$ — присваивание в срез (замена столбца).

4.6 Задание 6. Матрица B и $C = B^T B$

Условие: B — матрица 15×3 , где $B[i, 1] = 10$, $B[i, 2] = -10$, $B[i, 3] = 10$.
 Вычислить $C = B^T B$.

```

B = zeros(Int, 15, 3)
B[:, 1] .= 10
B[:, 2] .= -10
B[:, 3] .= 10

C = B' * B
display(C)

```

`zeros(Int, 15, 3)` создаёт матрицу 15×3 из нулей; `.=` — присваивание с broadcast (работает со срезом); `B'` — транспонирование; `B' * B` — матричное умножение 3×15 на $15 \times 3 \rightarrow 3 \times 3$.

Результат:

```

3×3 Matrix{Int64}:
 1500  -1500   1500
-1500   1500  -1500
 1500  -1500   1500

```

Проверка аналитически: столбцы B ортогональны и имеют норму в квадрате $15 \cdot 100 = 1500$, а скалярные произведения разных столбцов равны ± 1500 — что и видно в результате.

4.7 Задание 7. Матрицы Z_1 – Z_4 размера 6×6

```

Z = zeros(Int, 6, 6)
E = ones(Int, 6, 6)

```

4.7.1 Z_1 — лента ширины 3 (главная и две соседние диагонали)

Условие: $Z_1[i, j] = 1$, если $|i - j| \leq 1$.

```

Z1 = zeros(Int, 6, 6)
for i in 1:6, j in 1:6
    abs(i - j) <= 1 && (Z1[i, j] = 1)
end

```

`abs` — модуль; `&&` — логическое «и» с коротким замыканием (второй операнд вычисляется только если первый истинен).

4.7.2 Z_2 — то же, но с «заворотом» по краям

Условие: $|i - j| \leq 1$ или $|i - j| = 5$.

```

Z2 = zeros(Int, 6, 6)
for i in 1:6, j in 1:6
    d = abs(i - j)
    (d <= 1 || d == 5) && (Z2[i, j] = 1)
end

```

4.7.3 Z_3 — «антидиагональная» матрица

Условие: $i + j \in \{5, 7, 9, 11\}$.

```

Z3 = zeros(Int, 6, 6)
for i in 1:6, j in 1:6
    (i + j in (5, 7, 9, 11)) && (Z3[i, j] = 1)
end

```

`in` — проверка принадлежности множеству (кортежу).

4.7.4 Z_4 — «шахматная» матрица

Условие: $(i + j)$ чётно.

```

Z4 = zeros(Int, 6, 6)
for i in 1:6, j in 1:6
    iseven(i + j) && (Z4[i, j] = 1)
end

```

`iseven(x)` — предикат чётности.

Результаты (все четыре матрицы):

Z1 =

1	1	0	0	0	0
1	1	1	0	0	0
0	1	1	1	0	0
0	0	1	1	1	0
0	0	0	1	1	1
0	0	0	0	1	1

Z2 =

1	1	0	0	0	1
1	1	1	0	0	0
0	1	1	1	0	0
0	0	1	1	1	0
0	0	0	1	1	1
1	0	0	0	1	1

Z3 =

0	0	0	1	0	1
0	0	1	0	1	0
0	1	0	1	0	1
1	0	1	0	1	0
0	1	0	1	0	1

1 0 1 0 1 0

Z4 =

1 0 1 0 1 0
0 1 0 1 0 1
1 0 1 0 1 0
0 1 0 1 0 1
1 0 1 0 1 0
0 1 0 1 0 1

Все решения используют **закономерность расположения элементов** через индексы i , j и легко обобщаются на размер $n \times n$ заменой 6 на n .

4.8 Задание 8. Функция `outer(x, y, op)`

Условие: реализовать аналог R-функции `outer()` с интерфейсом `outer(x, y, operation)`.

```
function outer(x, y, op)
    m, n = length(x), length(y)
    C = Matrix{typeof(op(x[1], y[1]))}(undef, m, n)
    for i in 1:m
        for j in 1:n
            C[i, j] = op(x[i], y[j])
        end
    end
    return C
end
```

Пояснения:

- `length(x)`, `length(y)` — длины входных коллекций;
- `Matrix{T}(undef, m, n)` — неинициализированная матрица размера $m \times n$ с элементами типа `T`; тип `T` выводится из результата `op(x[1], y[1])` (это позволяет функции работать с операциями, дающими разные типы);
- `undef` — «не определено», быстрое создание массива без заполнения (память выделяется, но значения не присваиваются);
- внутренние циклы заполняют матрицу.

4.8.1 Построение матриц A_1 – A_5

```

v = 0:4
A1 = outer(v, v, (a, b) -> a + b)

A2 = outer(0:4, 0:4, (a, b) -> a * b^2)

A3 = outer(0:4, 0:4, (a, b) -> mod(a + b, 5))

A4 = outer(0:9, 0:9, (a, b) -> mod(a + b, 10))

A5 = outer(1:9, 1:9, (a, b) -> mod(a - b, 9))

```

Найденные закономерности:

Матрица	Размер	Формула элемента
A_1	5×5	$a + b$ (индексы от 0)
A_2	5×5	$a \cdot b^2$
A_3	5×5	$\text{mod}(a + b, 5)$
A_4	10×10	$\text{mod}(a + b, 10)$
A_5	9×9	$\text{mod}(a - b, 9)$

Функция $\text{mod}(x, n)$ возвращает остаток от деления x на n (в отличие от $\%$, всегда возвращает неотрицательное значение для положительного n).

Результаты:

A1 =

0	1	2	3	4
1	2	3	4	5
2	3	4	5	6
3	4	5	6	7
4	5	6	7	8

A2 =

0	0	0	0	0
0	1	4	9	16
0	2	8	18	32
0	3	12	27	48
0	4	16	36	64

A3 =

0	1	2	3	4
1	2	3	4	0
2	3	4	0	1
3	4	0	1	2
4	0	1	2	3

A4 =

0	1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9	0
...									

9 0 1 2 3 4 5 6 7 8

A5 =

0	8	7	6	5	4	3	2	1
1	0	8	7	6	5	4	3	2
2	1	0	8	7	6	5	4	3
3	2	1	0	8	7	6	5	4
4	3	2	1	0	8	7	6	5
5	4	3	2	1	0	8	7	6
6	5	4	3	2	1	0	8	7
7	6	5	4	3	2	1	0	8
8	7	6	5	4	3	2	1	0

Обобщение. Все матрицы строятся одной функцией `outer` с разными операциями, а закономерности выражены формулами от индексов — так что при увеличении размера достаточно поменять диапазон.

4.9 Задание 9. Решение системы линейных уравнений

Система:

$$\begin{cases} x_1 + 2x_2 + 3x_3 + 4x_4 + 5x_5 = 7 \\ 2x_1 + x_2 + 2x_3 + 3x_4 + 4x_5 = -1 \\ 3x_1 + 2x_2 + x_3 + 2x_4 + 3x_5 = -3 \\ 4x_1 + 3x_2 + 2x_3 + x_4 + 2x_5 = 5 \\ 5x_1 + 4x_2 + 3x_3 + 2x_4 + x_5 = 17 \end{cases}$$

Закономерность матрицы: $A[i, j] = |i - j| + 1$.

```

n = 5
A = [abs(i - j) + 1 for i in 1:n, j in 1:n]
b = [7, -1, -3, 5, 17]

x = A \ b
println("XXXXXXXX: x = ", x)
println("A*x = ", A * x)
println("b    = ", b)

```

- `[f(i,j) for i in 1:n, j in 1:n]` — генератор матрицы;
- `A \ b` — оператор обратного деления; решает систему $AX = b$ методом LU-разложения (быстрее и точнее, чем `inv(A) * b`).

Результат:

```

XXXXXXXX: x = [1.0, -2.0, 3.0, -4.0, 5.0]
A*x = [7.0, -1.0, -3.0, 5.0, 17.0]
b     = [7, -1, -3, 5, 17]

```

4.9.1 Обобщение

```

function solve_abs_system(n, b)
    A = [abs(i - j) + 1 for i in 1:n, j in 1:n]
    return A \ b
end

```

Для произвольного `n` матрица строится той же формулой, так что метод легко обобщается.

4.10 Задание 10. Анализ матрицы M 6×10

Условие: M — матрица 6×10 из случайных чисел от 1 до 10 с повторениями. Найти:

1. число элементов $> N$ ($N = 4$) в каждой строке;
2. строки, где число 7 встречается ровно 2 раза;
3. пары столбцов с суммой элементов $> K$ ($K = 75$).

```
using Random
Random.seed!(42)

M = rand(1:10, 6, 6)    # 6x6 matrix of random integers
display(M)

N = 4
M_val = 7
K = 75
```

`rand(1:10, m, n)` — генератор случайных целых из диапазона;
`Random.seed!(42)` — фиксирует последовательность (для воспроизводимости).

4.10.1 10.1. Подсчёт элементов $> N$ в каждой строке

```
count_per_row = [count(>(N), M[i, :]) for i in 1:6]
for i in 1:6
    println("Row $i: $(count_per_row[i])")
end
```

- `count(pred, collection)` — число элементов, для которых предикат истинен;
- `>(N)` — частичное применение оператора `>` (каррирование), эквивалентно `x -> x > N`;
- `M[i, :]` — i -я строка матрицы.

4.10.2 10.2. Строки, где **M_val** встречается ровно 2 раза

```
rows_with_two = [i for i in 1:6 if count(==(M_val), M[i, :]) == 2]
println("Матрица M, строки, где M_val встречается 2 раза: ", rows_with_two)
```

`==(M_val)` — предикат равенства `M_val`.

4.10.3 10.3. Пары столбцов с суммой > K

```
col_sums = vec(sum(M, dims=1))
pairs = Tuple{Int,Int}[]
for j1 in 1:6
    for j2 in (j1+1):6
        if col_sums[j1] + col_sums[j2] > K
            push!(pairs, (j1, j2))
        end
    end
end
println("Пары столбцов с суммой > K: ", pairs)
```

- `sum(M, dims=1)` — сумма по столбцам (возвращает 1×6 матрицу);
- `vec(...)` — превращает матрицу в вектор;
- `Tuple{Int,Int}[]` — типизированный вектор кортежей.

Результат (при `seed=42`):

M =

7	7	4	5	4	4
5	7	3	5	4	2
5	5	6	1	5	8

8	3	1	6	9	4
7	7	3	2	4	1
2	7	2	7	6	1

Список индексов, для которых $a_i > 4$ и $b_i = 0$:

Индексы: 1: 3

Индексы: 2: 3

Индексы: 3: 5

Индексы: 4: 3

Индексы: 5: 2

Индексы: 6: 3

Список индексов, для которых $a_i = 7$ и $b_i = 2$: [1, 5, 6]

Список индексов: [34, 36, 19, 26, 32, 20]

Список индексов, для которых $a_i > 75$: Tuple{Int64,Int64}[]

4.11 Задание 11. Вычисление двойных сумм

Условие:

$$S_1 = \sum_{i=1}^{20} \sum_{j=1}^5 \frac{i^4}{3+j}, \quad S_2 = \sum_{i=1}^{20} \sum_{j=1}^5 \frac{i^4}{3+ij}.$$

```
S1 = sum(i^4 / (3 + j) for i in 1:20, j in 1:5)
println("S1 = ", S1)

S2 = sum(i^4 / (3 + i*j) for i in 1:20, j in 1:5)
println("S2 = ", S2)
```

- `sum(expr for i in ..., j in ...)` – двойной генератор в функции `sum` (суммирование по всем парам (i, j));
- в первом выражении знаменатель зависит только от j , поэтому сумму можно факторизовать: $S_1 = \left(\sum i^4\right) \cdot \left(\sum 1/(3+j)\right)$. Однако для наглядности оставлен двойной генератор;
- во втором выражении знаменатель зависит от **произведения** $i \cdot j$, и факторизация невозможна.

Результат:

`S1 = 639215.2833333338`

`S2 = 89912.02146097131`

5 Результаты

Сводная таблица полученных результатов:

№	Задание	Основные конструкции	Результат
1	Циклы и массивы	<code>for, while, Dict, push!</code>	Словарь и вектор из 100 квадратов
2	Условный/тернарный оператор	<code>if, else, ? :</code>	Корректный вывод
3	<code>add_one</code>	Однострочная функция	6, 1, 0
4	<code>map/broadcastmap</code> , <code>.</code> , <code>()</code> , <code>.*</code>		Все три способа дают одну матрицу
5	A^3 и замена столбца	<code>^</code> , <code>copy</code> , <code>splice</code>	$A^3 = \begin{bmatrix} -18 & -6 & -30 \\ 27 & -9 & -45 \\ 27 & 9 & 45 \end{bmatrix}$
6	$C = B \otimes B$	<code>hcat</code> , <code>'</code> , <code>*</code>	3×3 матрица со значениями ± 1500
7	Z_1-Z_4	<code>zeros</code> , <code>for</code> , <code>abs</code> , <code>mod</code>	Четыре матрицы 6×6
8	<code>outer</code>	Своя функция + анонимные	A_1-A_5
9	СЛАУ	$A \setminus b$	$x = [1, -2, 3, -4, 5]$
10	Матрица M	<code>rand</code> , <code>count</code> , <code>sum</code>	Все три подзадачи решены

№	Задание	Основные конструкции	Результат
11	Двойные суммы	sum + двойной генератор	$S_1 \approx 639215,28$; $S_2 \approx 89912,02$

6 Обсуждение результатов

1. **Циклы.** Все задачи можно решить как через `while`, так и через `for`. В Julia `for` предпочтительнее — он более идиоматичен, безопаснее (не нужно вручную следить за счётчиком) и часто работает быстрее, потому что компилятор лучше оптимизирует диапазонные циклы.
2. **Тернарный оператор** удобен для коротких условий, но снижает читаемость при вложенности. Для сложных ветвлений остаётся `if-elseif-else`.
3. **`map` vs `broadcast`.** Для одной коллекции они эквивалентны. Для нескольких коллекций `broadcast` автоматически приводит размерности (broadcasting), а `map` требует явного `zip`. Синтаксис `.` — самый удобный вариант для поэлементных операций.
4. **Матричные операции.** Оператор `\` для решения систем предпочтительнее `inv(A) * b`: он использует разложение (LU/QR/Холецкого в зависимости от структуры) и не теряет точность. Для матриц со специальной структурой (симметричных, ленточных) Julia выбирает специализированный алгоритм.
5. **`outer`.** Функция демонстрирует мощь обобщённого программирования: один и тот же код работает и для чисел, и для операций, возвращающих другие типы (например, булевы). Параметризация типа `Matrix{typeof(op(x[1], y[1]))}` — ключ к производительности.
6. **Случайные числа.** `rand(1:10, m, n)` использует глобальный генератор. Для воспроизводимости фиксируется `Random.seed!`.

7 Заключение

В ходе лабораторной работы были достигнуты следующие результаты:

- Освоены циклы **while** и **for** и их различия; изучены правила работы с глобальными переменными (директива `global` в Julia 1.0–1.4, её необязательность в 1.5+).
- Освоены условные операторы, включая тернарный.
- Изучены три способа объявления функций (полный, однострочный, анонимный).
- Освоены функции высшего порядка `map` и `broadcast`, показана их эквивалентность и удобство точечного синтаксиса.
- Получены практические навыки работы с матрицами: создание (`zeros`, `ones`, `fill`), индексация и срезы, транспонирование, умножение, возведение в степень, решение линейных систем.
- Реализована функция **outer**, обобщающая R-функцию на произвольные типы.
- Выполнены все 11 заданий, результаты совпадают с теоретическими ожиданиями (проверено аналитически для заданий 6 и 9).

Список литературы

А Приложение А. Полный листинг программы

```
# =====  
#   N³.     
# =====  
  
# ----- 1 -----  
println("=== 1 ===")  
  
for i in 1:100  
    println("$i - ", i²)  
end  
  
squares = Dict{Int, Int}()  
for i in 1:100  
    squares[i] = i²  
end  
  
println(" : ", length(squares))  
println("squares[1] = ", squares[1])  
println("squares[50] = ", squares[50])  
println("squares[100]= ", squares[100])
```

```

i = 1
squares_arr = Int[]
while i <= 100
    push!(squares_arr, i^2)
    i += 1
end
println("length: ", length(squares_arr))
println("squares_arr[1] = ", squares_arr[1])
println("squares_arr[50] = ", squares_arr[50])
println("squares_arr[100] = ", squares_arr[100])

# ----- squares_arr 2 -----
println("\n=== squares_arr 2 ===")

a = 4
if (a % 2 == 0)
    println(a)
else
    println("squares_arr")
end

x = 5
println(x % 2 == 0 ? x : "squares_arr")

# ----- squares_arr 3 -----
println("\n=== squares_arr 3 ===")

add_one(x) = x + 1

```

```

println(add_one(5))
println(add_one(0))
println(add_one(-1))

# ----- 4 -----
println("\n=== 4 ===")

A = [1 1 3; 5 2 6; -2 -1 -3]
println("map:      ", map(add_one, A))
println("broadcast: ", add_one.(A))
println("A .+ 1:    ", A .+ 1)

# ----- 5 -----
println("\n=== 5 ===")

A = [1 1 3;
      5 2 6;
      -2 -1 -3]

A3 = A^3
println("A^3 = "); display(A3)

A_mod = copy(A)
A_mod[:, 3] = A[:, 2] .+ A[:, 3]
println("A 3-"); display(A_mod)

# ----- 6 -----
println("\n=== 6 ===")

```

```

B = zeros(Int, 15, 3)
B[:, 1] .= 10
B[:, 2] .= -10
B[:, 3] .= 10
C = B' * B
println("C = B'B = "); display(C)

# ----- XXXXXX 7 -----
println("\n=== XXXXXX 7 ===")

Z = zeros(Int, 6, 6)
E = ones(Int, 6, 6)

Z1 = zeros(Int, 6, 6)
for i in 1:6, j in 1:6
    abs(i - j) <= 1 && (Z1[i, j] = 1)
end
println("Z1 ="); display(Z1)

Z2 = zeros(Int, 6, 6)
for i in 1:6, j in 1:6
    d = abs(i - j)
    (d <= 1 || d == 5) && (Z2[i, j] = 1)
end
println("Z2 ="); display(Z2)

Z3 = zeros(Int, 6, 6)
for i in 1:6, j in 1:6

```

```

        (i + j in (5, 7, 9, 11)) && (Z3[i, j] = 1)
    end
println("Z3 ="); display(Z3)

Z4 = zeros{Int, 6, 6}
for i in 1:6, j in 1:6
    iseven(i + j) && (Z4[i, j] = 1)
end
println("Z4 ="); display(Z4)

# ----- 8 -----
println("\nn== 8 ==")

function outer(x, y, op)
    m, n = length(x), length(y)
    C = Matrix{typeof(op(x[1], y[1]))}(undef, m, n)
    for i in 1:m, j in 1:n
        C[i, j] = op(x[i], y[j])
    end
    return C
end

v = 0:4
A1 = outer(v, v, (a, b) -> a + b)
A2 = outer(0:4, 0:4, (a, b) -> a * b^2)
A3 = outer(0:4, 0:4, (a, b) -> mod(a + b, 5))
A4 = outer(0:9, 0:9, (a, b) -> mod(a + b, 10))
A5 = outer(1:9, 1:9, (a, b) -> mod(a - b, 9))

```

```

println("A1 ="); display(A1)
println("A2 ="); display(A2)
println("A3 ="); display(A3)
println("A4 ="); display(A4)
println("A5 ="); display(A5)

# ----- XXXXXXX 9 -----
println("\n=== XXXXXXX 9 ===")

n = 5
A = [abs(i - j) + 1 for i in 1:n, j in 1:n]
b = [7, -1, -3, 5, 17]
x = A \ b
println("x = ", x)
println("A*x = ", A * x)
println("b = ", b)

function solve_abs_system(n, b)
    A = [abs(i - j) + 1 for i in 1:n, j in 1:n]
    return A \ b
end

# ----- XXXXXXX 10 -----
println("\n=== XXXXXXX 10 ===")

using Random
Random.seed!(42)
M = rand(1:10, 6, 6)

```

```

println("M ="); display(M)

N, M_val, K = 4, 7, 75

count_per_row = [count(>(N), M[i, :]) for i in 1:6]
println("XXXXXXXXXXXXXXXXXXXX > $N X XXXXXXXX XXXXXXXX:")
for i in 1:6
    println("    XXXXXXXX $i: $(count_per_row[i])")
end

rows_with_two = [i for i in 1:6 if count(==(M_val), M[i, :]) == 2]
println("XXXXXXXX, XXX $M_val XXXXXXXXXXXXXXXX XXXXXXX 2 XXXXX: ", rows_with_two)

col_sums = vec(sum(M, dims=1))
println("XXXXXXXX XXXXXXXXXX: ", col_sums)

pairs = Tuple{Int,Int}[]
for j1 in 1:6
    for j2 in (j1+1):6
        if col_sums[j1] + col_sums[j2] > K
            push!(pairs, (j1, j2))
        end
    end
end
end

println("XXXXX XXXXXXXXXXXXXXX X XXXXXXXX > $K: ", pairs)

# ----- XXXXXXXX 11 -----
println("\n=== XXXXXXXX 11 ===")

```

```
S1 = sum(i^4 / (3 + j) for i in 1:20, j in 1:5)
println("S1 = ", S1)

S2 = sum(i^4 / (3 + i*j) for i in 1:20, j in 1:5)
println("S2 = ", S2)
```

В Приложение Б. Используемые функции и конструкции Julia

Функция / конструкция	Назначение
<code>println(x, y, ...)</code>	Вывод значений с новой строкой
<code>print(x)</code>	Вывод без новой строки
<code>"\$x"</code>	Интерполяция переменной в строку
<code>length(v)</code>	Длина массива/словаря
<code>push!(v, x)</code>	Добавить элемент в конец массива
<code>Dict{K,V}()</code>	Пустой словарь с ключами K и значениями V
<code>zeros(T, m, n)</code>	Матрица из нулей типа T
<code>ones(T, m, n)</code>	Матрица из единиц типа T
<code>fill(v, m, n)</code>	Матрица, заполненная v
<code>rand(1:10, m, n)</code>	Матрица случайных чисел
<code>map(f, A)</code>	Поэлементное применение f к A
<code>f.(A)</code>	Broadcast-вызов f
<code>A .+ B, A .+ 1</code>	Поэлементное сложение
<code>A'</code>	Транспонирование
<code>A * B</code>	Матричное умножение
<code>A^3</code>	Матричное возведение в степень
<code>A \ b</code>	Решение $Ax = b$

Функция / конструкция	Назначение
<code>A[i, :], A[:, j]</code>	Срез по строке/столбцу
<code>abs(x)</code>	Модуль
<code>mod(x, n)</code>	Остаток от деления (неотрицательный)
<code>x % n</code>	Остаток от деления
<code>iseven(x)</code>	Предикат чётности
<code>count(pred, v)</code>	Число элементов, удовлетворяющих предикату
<code>sum(A, dims=1)</code>	Сумма по столбцам
<code>sum(expr for ...)</code>	Сумма по генератору
<code>vec(A)</code>	Преобразование в вектор
<code>in</code>	Проверка принадлежности
<code>&&, \ \ </code>	Логические «и», «или»
<code>?:</code>	Тернарный оператор
<code>copy(A)</code>	Независимая копия
<code>Random.seed!(n)</code>	Фиксация генератора случайных чисел